

MLOPS IN PRODUCTION · ISSUE 06 OF 08

Feature quality and lineage.

Lineage answers the auditor's question in minutes, not weeks.

006

AR

Akhilesh Ranjan Singh

Senior DevSecOps Engineer · github.com/iarsingh

Swipe →

Checks before training

- ☒ Schema and types

- ☒ Null rate vs baseline

- ☒ Ranges and categories

- ☒ Freshness

- ☒ No future information (leakage)

- ☒ Train/serve skew

Leakage traps

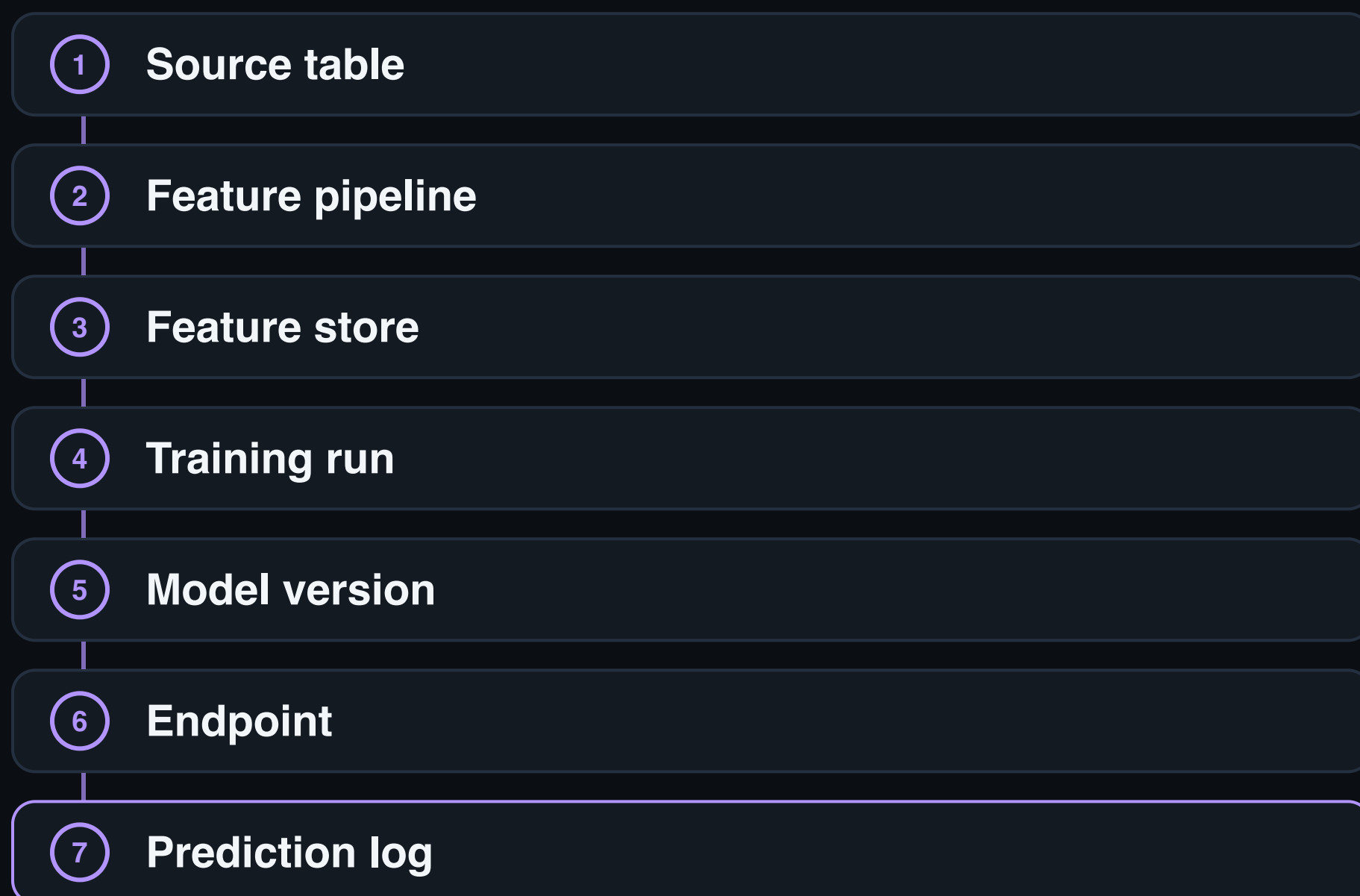
DO

- ✓ Use point-in-time joins
- ✓ Split by time for time-series
- ✓ Review features that look too good

AVOID

- ✗ Features computed after the label
- ✗ Random split on time data
- ✗ Target-encoded features from all data

Lineage graph



Every arrow is a recorded link, not a wiki page.

Questions lineage must answer

01 Which data?

That trained the model behind this prediction.

02 Which features?

And which version of their definitions.

03 Who approved?

And what evidence they saw.

04 Can we reproduce it?

From the recorded inputs alone.

Feature store governance

CONTROL	WHY
Owner per feature	Someone fixes it when it breaks
Definition in code	Reviewed and versioned
Freshness SLA	Stale features fail silently
Access control	PII stays restricted
Deprecation policy	Dead features get removed

KEY TAKEAWAY

Check features before training, trace them after deployment.

- Six quality checks
- Point-in-time joins prevent leakage
- Lineage from source to prediction

YOUR TURN

How long would it take you to trace a prediction back to its training data?

FOLLOW AKHILESH RANJAN SINGH

Next in the series: #07 Batch vs real-time serving

Save · Repost

